

FlockTrax Feed Type And BinSentry Order Logic Spec

Date: 2026-06-07

Workspace: C:\dev\FlockTrax

Status: design/spec only, not yet implemented

Purpose

This spec defines how FlockTrax should evolve from a total-feed projection tool into a feed-type-aware ordering system that can support:

- starter vs grower ordering logic
- FIFO bin-layer tracking
- type-aware use of BinSentry inventory
- future optional BinSentry write-back

The immediate business need is to support ordering decisions correctly when:

- live inventory is known from BinSentry
- feed deliveries are recorded in FlockTrax
- bins may contain accessible feed plus queued feed above it
- starter demand follows specific flock rules

Current Operational Facts

Feed types in use

Current business rule:

- only Starter and Grower are in use
- Finisher is not currently used

Starter consumption rule

Current business rule:

- each chick is expected to use 2.5 lbs of Starter

Baseline formula:

- `starter_target_lbs = placed_chicks * starter_lbs_per_chick`

Recommendation:

- make `starter_lbs_per_chick` an `app_setting`
- default it to 2.5

Possible future refinement:

- allow early mortality adjustment if that produces a better ordering estimate for additional starter

Existing system strengths

FlockTrax already knows:

- placements
- flock timing
- feed tickets
- feed drops
- feed type on delivered drops

BinSentry already provides:

- live pounds on hand by bin

Current gap

BinSentry inventory is quantity-aware, but not currently trusted by FlockTrax to be feed-type-aware.

Although BinSentry may support feed type in bins, that is not currently how operations are maintained. In practice, FlockTrax is where feed type is currently recorded on delivered drops.

FIFO qualifier

Important business rule:

- feed bins are operationally FIFO
- feed on top of another feed type cannot be consumed until the lower feed is fed out

That means a bin is not always a single-type bin.

A bin may contain:

- an accessible lower layer
- a queued upper layer

Example:

- 8,000 lbs accessible Starter
- 16,000 lbs queued Grower above it

In that case:

- the birds can only consume the Starter immediately
- the Grower is present, but not yet accessible

This makes FIFO inventory a layered-bin problem, not a simple current-type-only problem.

Recommended Source Of Truth Split

FlockTrax should be source of truth for

- feed layer / feed phase interpretation
- ordering logic
- order commitments
- received feed operations
- starter/grower business rules

BinSentry should be source of truth for

- live pounds on hand
- sensor-derived inventory readings

Why this split is preferred

- FlockTrax already captures the operational feed events
- BinSentry already proves useful for live quantity
- business rules are flock-driven, not sensor-driven
- this avoids having two systems compete over the same business meaning

Design Principles

1. Keep quantity and feed-layer meaning separate

- BinSentry gives quantity
 - FlockTrax determines what portion is accessible now and what portion is queued next
2. Keep ordering logic type-aware
 - starter inventory only offsets starter demand
 - grower inventory only offsets grower demand
 3. Model FIFO bins as layered inventory
 - do not force each bin into a single-state assumption
 - distinguish accessible feed from queued feed
 4. Forecast operational readiness, do not claim certainty
 - actual empty-bin outcome depends on worker rotation behavior
 - the system should forecast likely readiness, not guarantee it
 5. Allow later BinSentry write-back
 - only after FlockTrax-first logic is stable
 - only if the BinSentry API for this tenant exposes usable write actions

Required Data Model Changes

1. Feed bin layered feed state

Add to `public.feedbins`:

- `accessible_feed_type` text null
- `accessible_feed_lbs` numeric(12,2) null
- `queued_feed_type` text null
- `queued_feed_lbs` numeric(12,2) null
- `feed_state_effective_at` timestamp with time zone null
- `feed_state_source` text null

Expected feed values:

- starter
- grower

Suggested meaning:

- `accessible_feed_type`

- the feed type currently reachable by birds now
 - `accessible_feed_lbs`
- pounds currently believed to be reachable now
 - `queued_feed_type`
- the next feed type stacked above the accessible layer, if any
 - `queued_feed_lbs`
- pounds currently believed to exist in that queued layer
 - `feed_state_effective_at`
- when the current layered interpretation became effective
 - `feed_state_source`
- how the state was assigned, such as:
 - `manual`
 - `ticket_inferred`
 - `binsentry_sync`

Important note:

- this is intentionally a practical 2-layer model
- it is not a full arbitrary-depth layer engine
- because current operations only require support for Starter and Grower

2. Feed order commitment feed type

Add to `public.feed_order_commitments`:

- `feed_type` text null

Expected values:

- `starter`
- `grower`

Why:

- on-order feed must offset demand by type, not just by total pounds

3. Inventory snapshot layer context

Recommended addition to `public.feed_inventory_snapshots`:

- `accessible_feed_type` text null

- queued_feed_type text null

Expected values:

- starter
- grower

Why:

- it freezes what the bin was believed to contain at snapshot time
- later feed-layer changes on the bin do not erase historical meaning

Alternative:

- leave snapshots without layer fields and always read from feedbins

Recommendation:

- prefer storing layer context on snapshot rows once the FIFO bin model is in place

Initial Backfill Strategy

Goal

Bootstrap feed-bin layered state without requiring full manual setup.

Recommended backfill rule

For each feed bin:

- use the most recent delivered FlockTrax feed drop assigned to that bin
- combine that with current BinSentry live pounds
- initialize:
 - accessible_feed_type
 - accessible_feed_lbs
 - queued_feed_type
 - queued_feed_lbs

conservatively

Initial practical rule:

- if current BinSentry pounds are effectively zero, set the bin as empty:

- accessible_feed_type = null
- accessible_feed_lbs = 0
- queued_feed_type = null
- queued_feed_lbs = 0
- otherwise treat the last delivered feed type as the accessible type first
- do not attempt to infer a queued layer unless there is enough operational evidence to support it

Set:

- feedbins.feed_state_effective_at
- feedbins.feed_state_source = 'ticket_inferred'

Why this is the safest first pass

- the latest fill event is the strongest current signal
- it is better than pretending full layered historical certainty exists where it does not

Ongoing Feed Layer Maintenance Rule

After backfill, FlockTrax should keep bin feed-layer state current.

Feed ticket rule

When a delivered feed drop is recorded with:

- feed_bin_id
- feed_type

Then update that bin:

- if the bin is effectively empty, the delivered feed becomes the accessible layer
- if the bin already contains accessible feed, the new delivered feed may become the queued layer
- update:
 - accessible_feed_type
 - accessible_feed_lbs
 - queued_feed_type
 - queued_feed_lbs
 - feed_state_effective_at
 - feed_state_source

Why

- the delivery event is the real operational phase-change trigger
- users are already capturing this information in FlockTrax

Ordering Logic Changes

Current logic

Current high-level model is effectively:

- total requirement
- minus total on hand
- minus total on order
- equals recommended order

This is not sufficient once starter and grower must be separated.

Proposed logic

All ordering calculations should become type-aware.

For each placement / barn:

- starter_requirement_lbs
- grower_requirement_lbs
- starter_accessible_on_hand_lbs
- grower_accessible_on_hand_lbs
- starter_queued_on_hand_lbs
- grower_queued_on_hand_lbs
- starter_on_order_lbs
- grower_on_order_lbs

Then calculate:

- $\text{starter_recommended_order_lbs} = \text{starter_requirement_lbs} - \text{starter_accessible_on_hand_lbs} - \text{starter_on_order_lbs}$
- $\text{grower_recommended_order_lbs} = \text{grower_requirement_lbs} - \text{grower_accessible_on_hand_lbs} - \text{grower_on_order_lbs}$

Clamp both at zero.

Totals can still be shown as:

- starter + grower

Queued inventory should be visible in reporting, but not treated as immediately consumable.

Starter-specific business rule

Starter need should not be inferred only from bin contents.

It should be governed by placement-level obligation.

Placement starter target

Baseline:

- $\text{starter_target_lbs} = \text{placed_chicks} * \text{starter_lbs_per_chick}$

Placement starter progress

Track or derive:

- starter_delivered_lbs
- starter_accessible_on_hand_lbs
- starter_queued_on_hand_lbs
- starter_on_order_lbs
- starter_remaining_need_lbs

Important rule

Grower inventory must not offset starter demand.

Only:

- starter accessible inventory
- starter orders

should offset starter need.

Relationship Between Placement Phase And Bin FIFO State

These are related, but not identical.

Placement-level phase

Represents:

- whether the flock still needs starter
- whether it has moved into grower consumption

Bin-level FIFO state

Represents:

- what feed type is currently accessible in a bin
- what feed type is queued above it, if any

Why both are needed

A placement may still have starter need, but:

- one bin could have accessible starter
- that same bin could also have queued grower above it
- another bin could already have accessible grower

So the ordering engine must consider:

- flock need by type
- accessible inventory by type
- queued inventory by type

Barn-Level Starter Readiness Rule

This rule applies at the barn level, not the individual-bin level.

Core business rule

At least one bin in the barn must be projected to be capable of receiving starter by the end of the current placement.

This is the correct timing anchor because:

- nothing is consumed between placements
- if a bin is not starter-capable at the end of the current placement, it will not become starter-capable before the next flock arrives without operational intervention

Important qualifier

This is a projected readiness rule, not a guaranteed truth.

Actual success depends on:

- correct bin rotation by farm workers
- intentionally leaving the needed bin open and feeding it down
- not blocking that bin with queued grower that the next flock cannot consume

Why this matters

The next flock must be able to receive Starter.

The next flock cannot consume Grower at startup because pellet size is too large.

Therefore, even if total-feed math looks acceptable, the barn is operationally at risk if no starter-capable bin is projected to be available by the end of the current placement.

Operational interpretation

FlockTrax should forecast:

- projected starter-bin readiness at current placement end

It should not claim guaranteed readiness.

Additional insight

Better forecasting should reduce this risk because lower excess feed on hand makes it easier for farm workers to rotate bins correctly and achieve a starter-capable bin at flock turn.

Reporting Impact

The 10-Day Feed Requirements report should evolve from total-only to type-aware views.

Recommended future display

For each barn / placement:

- starter requirement
- grower requirement
- starter accessible on hand

- grower accessible on hand
- starter queued on hand
- grower queued on hand
- starter on order
- grower on order
- starter recommended order
- grower recommended order
- total recommended order
- projected starter-bin readiness at current placement end

Reason

This preserves the operator's ability to see total pounds while also making the actual order decision correct.

BinSentry Integration Direction

What should happen now

Use BinSentry as a read-side quantity source only.

Meaning:

- pull live pounds on hand
- map those pounds into FlockTrax's layered FIFO interpretation

What may happen later

If BinSentry API write actions are confirmed for this tenant, future sync may include:

1. create/update feed orders from FlockTrax into BinSentry
2. mark BinSentry orders received when FlockTrax receives feed
3. optionally sync current layered feed state in each bin back to BinSentry

Recommendation on write-back timing

Do not make this the first implementation step.

First stabilize:

- FlockTrax FIFO bin-state model
- starter/grower-aware ordering math

- type-aware inventory rollups

Then evaluate BinSentry write-back as phase 2.

Practical Implementation Sequence

Phase 1. Schema

Add:

- `feedbins.accessible_feed_type`
- `feedbins.accessible_feed_lbs`
- `feedbins.queued_feed_type`
- `feedbins.queued_feed_lbs`
- `feedbins.feed_state_effective_at`
- `feedbins.feed_state_source`
- `feed_order_commitments.feed_type`
- optionally layer context on `feed_inventory_snapshots`

Phase 2. Backfill

Backfill feed-bin layered state from:

- latest delivered feed drop per bin
- current BinSentry live pounds

Phase 3. Operational update rule

On feed ticket save:

- if a drop has `feed_bin_id` and `feed_type`
- update the target bin's layered FIFO state

Phase 4. Inventory rollups

Split barn inventory into:

- starter accessible on hand
- grower accessible on hand
- starter queued on hand
- grower queued on hand

based on layered bin state

Phase 5. Order commitments

Split on-order calculations into:

- starter on order
- grower on order

Phase 6. Placement starter obligation

Implement:

- $\text{placed_chicks} * \text{starter_lbs_per_chick}$

as the baseline starter obligation rule

Phase 7. Report / dashboard updates

Update:

- dashboard popup
- 10-Day Feed Requirements report

to show:

- starter/grower-specific order logic
- accessible vs queued inventory
- projected starter-bin readiness at current placement end

Phase 8. Optional BinSentry write-back

Only after confirming live writable API actions for:

- order creation
- order update
- bin feed-state update

Open Questions

1. Should starter obligation stay fixed at $\text{starter_lbs_per_chick} * \text{placed_chicks}$, or should early mortality reduce it automatically?

2. Should a manual override exist on each bin for layered feed state?
3. Should a placement-level phase flag be stored explicitly, or derived from delivered starter vs target starter?
4. Is the 2-layer FIFO model sufficient, or do real operations require deeper layer history?

Recommendation Summary

Best near-term architecture:

- FlockTrax owns feed layer interpretation and ordering logic
- BinSentry owns live quantity
- FIFO feed layers become explicit in FlockTrax
- starter and grower calculations are separated
- accessible and queued inventory are separated
- barn-level projected starter-bin readiness is tracked at the end of the current placement
- BinSentry write-back is deferred until the FlockTrax-first model is stable

This is the safest path and fits how operations are already being recorded today.